E-commerce Sales Analysis Guide

2025-10-07

Contents

1 Setting Up Our Workshop (The Imports)

The Goal

To get all our necessary toolkits ready to use in our notebook.

The Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style('whitegrid')
```

The Detailed Explanation

import pandas as pd: Pandas is the most important tool for a data analyst. Think of it as a magical, super-powered spreadsheet program (like Excel or Google Sheets) but inside Python. It lets us organize our data into clean tables called DataFrames. We give it the nickname pd so we don't have to type pandas every time.

import numpy as np: NumPy is the master mathematician of Python. It's incredibly fast at handling lists of numbers and performing calculations. Pandas actually uses NumPy behind the scenes to

do its work. We give it the nickname `np`.

import matplotlib.pyplot as plt: This is our main artist. It's a library that lets us draw all kinds of charts and graphs. We import its `pyplot` module, which contains all the drawing tools, and give it the nickname `plt`.

import seaborn as sns: Seaborn is like a talented assistant to our main artist, Matplotlib. It helps us create more beautiful and complex charts with less code. It has better default colors and styles. We give it the nickname `sns`.

`sns.set_style('whitegrid')`: This is a command from Seaborn that sets a nice visual style for all the charts we're about to make. `'whitegrid'` gives our charts a clean white background with gray grid lines, making them look professional.

2 Loading the Evidence (Reading the Data)

The Goal

To load the data from the `online_retail.xlsx` file into a Pandas DataFrame so we can start working with it.

The Code

```
df = pd.read_excel('online_retail.xlsx', parse_dates=['InvoiceDate'])
```

The Detailed Explanation

`df = ...`: We are creating a variable called `df` (short for DataFrame) that will hold all our sales data.

`pd.read_excel(...)`: This is the specific Pandas command used to read data from a Microsoft Excel file (.xlsx).

`'online_retail.xlsx'`: This is the name of the file we want to read. It must be located in the same place as our code (or we must provide the full path to it).

`parse_dates=['InvoiceDate']`: This is a very powerful and smart command. We are telling Pandas ahead of time: "Hey, when you look at the `InvoiceDate` column, don't treat it as just plain text. Recognize that it contains actual dates and times, and convert it into a special, intelligent date format."

Why is this so important? By doing this now, we can easily ask questions later like "What month was this sale?" or "What day of the week was this?" without any extra work.

3 The Big Cleanup (Data Cleaning)

The Goal

To find and fix all the errors and missing information in our data to make it accurate and reliable.

The Code

```
# First, let's get a summary to find problems
df.info()

# --- Cleaning Actions ---

# 1. Drop rows with a missing CustomerID
df.dropna(subset=['CustomerID'], inplace=True)

# 2. Remove returns (where Quantity is negative)
df = df[df['Quantity'] > 0]

# 3. Remove items with a price of 0
df = df[df['UnitPrice'] > 0]

# 4. Change CustomerID to a whole number
df['CustomerID'] = df['CustomerID'].astype(int)
```

The Detailed Explanation

`df.info()`: This is our first diagnostic tool. It gives us a report on our DataFrame, showing each column's name, how many non-empty cells it has, and its data type (Dtype). This is how we discovered that `CustomerID` had many missing values.

1. **Handling Missing CustomerID**

Why? Our mission is to understand customer behavior. If a sale has no `CustomerID`, we don't know who bought the item. That row is useless for our customer analysis. It's like a library book was returned, but the librarian forgot to write down who borrowed it. We can't analyze that person's reading habits.

How? `df.dropna(subset=['CustomerID'], inplace=True)` is the command to fix this.

`dropna`: This means "drop Not Available," or in simple terms, "delete rows with empty cells."

`subset=['CustomerID']`: We don't want to delete a row if any cell is empty. We only care if the `CustomerID` cell is empty. `subset` tells Pandas to only look in this specific column.

`inplace=True`: This tells Pandas to make the change permanent on our DataFrame `df`.

2. Removing Negative Quantities

Why? In this dataset, a negative `Quantity` (like -1) means a customer returned an item. This isn't a sale; it's an "un-sale." If we include these, they will cancel out our real sales and give us wrong totals.

How? `df = df[df['Quantity'] > 0]` is a filtering command. Let's break it down from the inside out:

`df['Quantity'] > 0`: This part goes through the `Quantity` column and asks for each row, "Is the quantity greater than 0?" It creates a long list of `True` and `False` answers.

`df[...]`: We then use this list of `True`/`False` as a filter for our main DataFrame `df`. It only keeps the rows that got a `True` answer.

3. Removing Zero Prices

Why? An item with a `UnitPrice` of 0 is not a real sale. It could be a promotional giveaway or a data entry error. To calculate sales accurately, we must remove these.

How? We use the exact same filtering logic as before: `df = df[df['UnitPrice'] > 0]`.

4. Fixing the CustomerID Data Type

Why? Pandas initially read the CustomerIDs as float numbers (like 12345.0). Customer IDs should be clean, whole numbers (`int`), like 12345. This is a small but professional step to ensure our data is in the correct format.

How? `df['CustomerID'].astype(int)` is the command to convert all the values in that column to the `int` (integer) data type.

4 Creating New Clues (Feature Engineering)

The Goal

To create new columns that will make our analysis easier and more powerful.

The Code

```
df['TotalPrice'] = df['Quantity'] * df['UnitPrice']

df['Year'] = df['InvoiceDate'].dt.year
df['Month'] = df['InvoiceDate'].dt.month
df['DayOfWeek'] = df['InvoiceDate'].dt.day_name()
df['Hour'] = df['InvoiceDate'].dt.hour
```

The Detailed Explanation

`df['TotalPrice'] = df['Quantity'] * df['UnitPrice']`: We're creating a new column called `TotalPrice`. For each row, the value in this column is calculated by multiplying the `Quantity` by the `UnitPrice`. This is the most important clue for understanding the value of sales.

Extracting Date Parts: Remember how we used `parse_dates` in the beginning? This is where it pays off! Because Pandas knows `InvoiceDate` is a special date object, we can use the `.dt` accessor to easily pull out its parts.

- `.dt.year` gives us the year of the sale.
- `.dt.month` gives us the month (1 for Jan, 2 for Feb, etc.).
- `.dt.day_name()` gives us the name of the day ('Monday', 'Tuesday', etc.).

- `.dt.hour` gives us the hour of the sale (0-23).

Why do this? This turns a single date column into four new, powerful columns that we can use to group our data and find patterns, like ”Which day of the week has the most sales?”

5 Finding and Handling Troublemakers (IQR Outlier Detection)

The Goal

To identify and remove extreme, potentially misleading sales values from our `TotalPrice` column using a statistical method called the Interquartile Range (IQR).

The Detailed Explanation of the IQR Formula and Method

What’s an Outlier?

Imagine your class takes a test. Most students score between 70 and 90. But one student, a super-genius, scores 150 (the test had bonus questions!). Another student accidentally filled in their name and nothing else, scoring 0. Both the 150 and the 0 are outliers. If we calculate the average score, these two extremes will pull the average up or down, giving us a misleading idea of how the ”typical” student

performed.

How do we find them fairly? The IQR Method.

Instead of just guessing, we use a method based on ”quartiles.”

1. **Line Up Your Data.** Imagine we take every single `TotalPrice` from our data and line them up in order, from the smallest sale to the largest.
2. **Find the Quartiles.** We then divide this long line into four equal parts. The points where we divide are called quartiles.
   - **Q1 (1st Quartile):** This is the number that is 25% of the way through our sorted list. 25% of all sales are smaller than this value.
   - **Q2 (2nd Quartile):** This is the number that is 50% of the way through the list. This is also called the median.
   - **Q3 (3rd Quartile):** This is the number that is 75% of the way through our sorted list. 75% of all sales are smaller than this value.
3. **Calculate the IQR.** The Interquartile Range (IQR) is the range that covers the ”middle 50%” of all our sales. It’s the box in a boxplot. The formula is simple:

$$IQR = Q3 - Q1$$

This IQR value tells us the spread of the ”typical” or ”normal” sales.

4. **Build the Fences.** Now we create our ”fences.” Any value that falls outside these fences is considered an outlier.

$$LowerBound = Q1 - 1.5 \times IQR$$

$$UpperBound = Q3 + 1.5 \times IQR$$

Why 1.5? This is a standard rule in statistics, established by John Tukey. It’s considered a good balance—wide enough to include normal variations but strict enough to catch truly extreme values.

The Code

```
Q1 = df['TotalPrice'].quantile(0.25)
Q3 = df['TotalPrice'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Create a new DataFrame without the outliers
df_no_outliers = df[(df['TotalPrice'] >= lower_bound) & (df['TotalPri
```

The Code Explained

`df['TotalPrice'].quantile(0.25)`: This is the Pandas command to calculate the 25th percentile (Q1). We do the same for 0.75 to get Q3.

$$IQR = Q3 - Q1$$

We then calculate our `lower_bound` and `upper_bound` using the formulas.

`df_no_outliers = df[...]`: Finally, we use our filtering technique again. The condition `(df['TotalPrice'] >= lower_bound) & (df['TotalPrice'] <= upper_bound)` tells Pandas to keep only the rows where the `TotalPrice` is inside our fences. We save this cleaner data into a new DataFrame called `df_no_outliers`.

6 Finding Our Best Customers (RFM Analysis)

The Goal

To calculate the Recency, Frequency, and Monetary value for each customer to see who our VIPs are.

The Code

```
snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)

rfm = df.groupby('CustomerID').agg({
    'InvoiceDate': lambda date: (snapshot_date - date.max()).days,
    'InvoiceNo': 'nunique',
    'TotalPrice': 'sum'
})

rfm.rename(columns={'InvoiceDate': 'Recency', 'InvoiceNo': 'Frequency
```

The Detailed Explanation

`snapshot_date`: To calculate recency, we need a "today" to measure from. Since the data is old, we can't use today's actual date. So, we create a pretend "today" that is one day after the very last sale in our dataset. `pd.Timedelta(days=1)` is just a way to add one day to a date.

`df.groupby('CustomerID').agg({...})`: This is the most advanced command, but let's break it down.

- `.groupby('CustomerID')`: This is the first step. It's like taking a huge pile of papers (all our sales) and sorting them into small stacks, where each stack belongs to just one customer.
- `.agg({...})`: `agg` is short for aggregate. It means we want to

run some calculations on each of these small stacks (each customer's group of sales). We tell it what to calculate inside the curly braces {}.

- `'InvoiceDate': lambda date: ...`: For the `InvoiceDate` column in each stack, we perform a special calculation. `lambda date: ...` is a mini, on-the-spot function. It says, "Take all the dates for this one customer (`date`), find the latest one (`date.max()`), and subtract it from our `snapshot_date`." This gives us the **Recency**.
- `'InvoiceNo': 'nunique'`: For the `InvoiceNo` column, count the number of novel or unique invoice numbers. This tells us how many separate shopping trips the customer made. This is **Frequency**.
- `'TotalPrice': 'sum'`: For the `TotalPrice` column, add them all up. This is **Monetary Value**.

`rfm.rename(...)`: After the calculation, the columns have generic names. We use our rename command to give them the proper RFM names: `Recency`, `Frequency`, and `MonetaryValue`.

This concludes the Python part of our investigation. We have successfully taken a messy, complex dataset and transformed it into clean, insightful tables that are ready to be visualized in a dashboard.